

Self-Play Regression Harness for Pokerbots:

Approaches & Framework

Executive Summary

- **Objective:** Establish an offline self-play **match harness** to pit the current pokerbot against prior versions and fixed opponents, yielding reliable metrics to prioritize improvements. This harness must run **locally (no internet)** within strict constraints (no background pondering, memory/cpu limits, Python 3.7 target).
- **Key Requirements:** The framework should ensure **fair and deterministic** comparisons (e.g. fixed random seeds for reproducible deals), robust isolation between bots (preventing unintended info sharing), and thorough logging for debugging and performance analysis.
- **Approach Options:** We identify **three viable architectures** – (1) an **external process orchestrator** (engine spawns each bot as a separate process), (2) an **in-process simulation** (bots integrated into a single engine instance), and (3) a **client-server or containerized setup** (bots connect to a local game server or run in isolated containers). Each has trade-offs in reliability, complexity, and resource overhead.
- **Metrics for Evaluation:** To judge improvements, the harness will track **win rate, average chip EV per hand, variance**, and even **exploitability** against fixed strategies. Statistical rigor is crucial – e.g. hundreds of thousands of hands may be needed to confirm small win-rate differences due to poker variance ¹. We can incorporate techniques like **duplicate matches** (same deals with players swapping seats) to reduce variance and ensure fair comparisons ².
- **Deterministic Testing:** The system must support **deterministic replays** for regression testing – by controlling the RNG seed in the game engine so the same sequence of cards and events can be reproduced ³. This ensures a bug fix or code change doesn't alter outcomes except through improved decisions.
- **Security & Isolation:** To prevent cross-bot interference or cheating, bots should be **sandboxed** (separate memory space, no shared global state). The harness will use OS process isolation or containers so each bot only communicates via the official game interface ⁴. All external access (internet, file I/O aside from logs) remains disallowed per competition rules.
- **Failure Mitigation:** The harness must gracefully handle common failure modes – e.g. bot timeouts, deadlock waiting for input, or crashes. Time controls (like a 30s total clock per 1000-hand match ⁵) will be enforced to reset or forfeit stuck bots. I/O is managed with non-blocking reads/writes or threads to avoid pipe buffering deadlocks. Any bot process that hangs or exceeds memory can be terminated and recorded as a loss, ensuring the test run continues.
- **Integration into Development Cycle:** Building this harness is top priority (Priority 0) – deliverables include the match-runner script, versioned bot snapshots, a results dashboard, and updates to the plan.md. The harness results will directly inform the plan's priority ranking: e.g. automatically flag regressions or validate that a new strategy version beats the baseline with statistical significance. To avoid violating "no pondering", any strategy tuning or learning from these results happens **offline** – the submitted bot remains fixed during actual matches (no on-the-fly learning).

- **Recommended Strategy:** Use the **official Pokerbots engine** (or an adapted version of it) as the core, leveraging its proven match management and fairness features. This engine already supports multi-bot matches with configurable rounds, blinds, and timeouts ⁶ ⁵. By extending it with an automated runner (to loop matches and aggregate stats) and using separate bot process executions, we achieve a **robust, competition-faithful harness**. The other approaches (in-process or custom server) can be useful for quick checks or special cases, but the process-based engine harness offers the best balance of reliability, security, and fidelity to the real tournament conditions.

Approaches for Self-Play Harness Architecture

To run bot-vs-bot matches offline, we consider three main architecture approaches. Each approach can fulfill the core needs (running repeated poker games between different bot versions and logging results), but they differ in reliability, complexity, and how well they address the key questions (determinism, fairness, isolation, etc.). Below we describe each approach and its pros/cons:

Approach 1: External Process Orchestrator (Engine-Driven)

Description: Use a dedicated **game engine** program or script to run matches, launching each pokerbot as a **separate subprocess**. This is analogous to the official competition setup – the engine acts as dealer and referee, communicating with each bot via standardized I/O (stdin/stdout pipes or sockets). For example, the MIT Pokerbots engine spawns two bot processes given their file paths, then runs a full match of a fixed number of hands ⁶, enforcing time and memory rules. In our case, we can use the provided engine (or a modified version) to pit the current bot vs. a historical bot instance or a fixed AI.

Benefits: This architecture offers **strong isolation** – each bot runs in its own process, so a crash or memory leak in one won't directly take down the other or the engine. It prevents bots from sharing memory or internal state (they only exchange moves via the engine), addressing security and “cross-pollination” concerns. In fact, the Pokerbots engine was explicitly designed to handle bot subprocesses safely ⁴. Multi-language support is natural (the engine can launch a Python bot, a C++ bot, etc., as in competition). It also mirrors the real tournament conditions closely, which is ideal for realistic regression testing. Timeouts and resource limits can be enforced at the process level – e.g. the engine tracks each bot's cumulative decision time and will cut off a bot that exceeds the 30s game clock ⁵. From a **determinism** standpoint, the engine can be given a specific random seed for dealing (or we can modify it to accept one) to ensure reproducible sequences of cards – run the same match twice and you get identical outcomes if the bots' logic hasn't changed ³. Logging is typically built-in: the engine can record all game actions to a file (e.g. a `gameLog` with all bets, folds, outcomes) ⁷, and even each bot's stdout/stderr can be captured for debugging. This provides a **comprehensive trace** for attributing performance differences or diagnosing errors.

Drawbacks: The main downsides are slightly higher complexity in implementation and performance overhead. Managing subprocesses means dealing with I/O pipes or sockets – without care, one could hit deadlocks if buffers fill or if a bot doesn't read input. (For instance, writing too much to a bot's stdin without it reading could stall; we mitigate this by using threads or async I/O to continuously read bot outputs and keep pipes clear.) We must also handle process termination (kill a bot that freezes or exceeds limits). While the engine approach is robust, it's not “lightweight” in the sense that launching a process for each bot and coordinating them has overhead – but in practice this is usually minor relative to game simulation time. Another consideration is **version compatibility**: if the engine or game rules have changed over time, running an old bot binary with the new engine might require shims. We should store historical bots in a way

that they can still be executed (e.g. Docker images or a preserved environment if dependencies differ). If all versions use the same interface (e.g. the engine's API didn't change), this is not a big issue. To ensure fairness, we must also be careful that each match uses identical rules – using the same engine config (stack sizes, blind structure, number of rounds) for all comparisons. Overall, the process-driven harness is the **most reliable and faithful** approach, at the cost of some implementation effort in orchestration.

Approach 2: In-Process Integrated Simulation

Description: Embed both bots and the game logic *within a single process*. This could mean calling the engine's methods or using a library to simulate poker rounds by directly invoking each bot's decision function. For example, a Python framework like *PyPokerEngine* allows registering multiple AI players in code and running a game with them via function calls (no separate OS processes) ⁸. In our scenario, if the bots are coded in Python, we might import two versions of the bot (e.g. current and an old snapshot as modules) and have a driver function alternate between them to play a hand. Essentially, the harness acts as the dealer, calling `bot.action(state)` for each decision.

Benefits: This approach is **lightweight and fast** – there's no inter-process communication overhead or context switching. Everything runs in one Python interpreter (or one JVM, etc., if Java). It simplifies debugging: you can step through the simulation in a debugger, or directly inspect a bot's internal state during runtime (since it's not isolated). Logging can be as simple as appending to in-memory structures or printing – since the control is in one place, collecting data is straightforward. Deterministic replay is also straightforward: by seeding the RNG at the start and not introducing any external non-determinism, you can recreate the same hand sequence and decisions. This method is great for quick iteration and unit-testing of specific scenarios (you could even simulate a single hand or specific card deals to see how both versions respond).

Drawbacks: **Lack of isolation** is the biggest concern. Because both bots share a process, a malfunction in one (e.g. an infinite loop or a memory-intensive operation) can crash or slow down the entire simulation. There's also a risk of unintended interference: if not carefully managed, they might share global variables or singletons (for example, if the engine logic or bot code has any global state, one bot's actions might affect the other). We also lose the ability to easily run bots written in different languages – this approach typically assumes a single language environment. Security/privacy is weaker: a malicious or buggy bot could, in theory, introspect the process memory to glean the other's cards or strategy (though in our controlled setting we assume bots are not adversarial in that way, it's still less secure than separate processes). Implementing the full game logic in-process can also be non-trivial – we'd either need to reuse the engine's code as a library (if possible) or reimplement rules for dealing, betting rounds, showdown, etc., especially since our variant ("Toss or Hold'em") might have unique mechanics. Without careful coding, it's easy to introduce discrepancies from the real rules. Time management is also on us – we don't have an external clock enforcement, so we'd have to implement timers to simulate the competition clock (e.g. track how long each bot is thinking and break if it exceeds the limit). This is doable (Python's `signal` alarm or threading timers), but adds complexity. In summary, the in-process method trades robustness for simplicity. It is **best used for small-scale testing** and rapid prototyping, but for official regression benchmarks, it might not provide the confidence we need, given the risk that a bot's code could break the harness or that differences from the official engine could skew results.

Approach 3: Client-Server or Containerized Matchmaking

Description: Set up a dedicated **match server** that bots connect to, potentially running each bot in an isolated environment (container or sandbox). This is similar to how the Annual Computer Poker Competition (ACPC) server works: a central **dealer server** listens on sockets, and each pokerbot runs as a client process that connects and plays the game via a protocol ⁹ ². In a modern twist, one could run each bot inside a Docker container or VM to fully sandbox their filesystem and processes, communicating only through a local network or IPC. The server would deal cards, enforce rules, send each player the game state updates, and receive their actions.

Benefits: This approach offers the **highest level of isolation and scalability**. Each bot could run on a separate machine entirely, or with resource constraints applied (CPU limits, memory limits via container config), ensuring one cannot impact the performance of the other. It also mirrors a production or competitive server environment, which is useful if we expect to later deploy our bot in such a setting. Security is strong: bots only interact through socket messages as defined by the poker protocol – no shared memory or file access. If needed, this can prevent any possibility of information leakage or cheating beyond the protocol (for instance, you could even monitor that one bot isn't sending covert signals by timing or such – though on one machine that's less a concern). Another advantage is that a well-defined protocol (like the ACPC protocol ⁹) can support a variety of game variants; if such a server is available, it might handle the dealing logic and match orchestration out-of-the-box. Using containers for each bot also means you can easily run old versions in the exact environment they were built for, avoiding dependency or Python version issues – each snapshot bot could be a container image tagged with its version.

Drawbacks: Complexity and overhead are significant here. Setting up a full client-server system requires additional infrastructure – you need to run the server program, possibly configure networking (ports etc.), and ensure clients connect reliably. There's potential for connection issues or **port conflicts** if running many matches (though on localhost this is manageable by using different ports or sequential runs) – e.g., if a socket isn't closed properly and you try to reuse the port, you could see address-in-use errors. We'd also have to implement or adopt a poker server; unless the official engine can operate in a server mode, this might mean writing a custom server or using an open-source one. That's a lot of work compared to simply spawning processes. The network layer also introduces a small overhead in communication and complexity in error handling (e.g., what if a bot client disconnects mid-game due to a crash? The server needs to detect that and handle it as a loss or restart). Debugging can be harder because the logic is split – you would have to look at server logs and bot logs separately. Running everything on one machine via containers also consumes more resources (each container duplicates some overhead) – given our resource limits, this could be overkill. In short, while this approach maximizes fidelity and isolation, it **overshoots the needs of an offline regression test harness** in most cases. It might be justified if we wanted to simulate the exact conditions of an official competition server or run many matches in parallel across machines, but otherwise the added complexity likely outweighs the benefits for our use case.

Other Considerations Across Approaches

Regardless of approach, there are several **common considerations** to address:

- **Reproducibility:** All approaches should allow seeding the random number generator that shuffles cards. In Approach 1 or 3, this might mean adding a seed option to the engine/server (or using a fixed seed internally). In Approach 2, we can directly seed the PRNG in code ³. This is crucial for

deterministic replays (Question 4) – if a new bot version loses a specific hand badly, we want to replay that same hand sequence to see what went wrong once we fix the bot. Ensuring each match or series has a known seed (logged with the results) will allow exact regression testing.

- **Fair Match Format:** We should configure matches to be as fair and statistically stable as possible (Question 5). This means ensuring equal opportunity for positional advantage. In heads-up poker, being dealer (or small blind) confers an edge, so fairness demands *seat rotation*. The official approach is often to play an even number of hands and alternate dealer each hand, or use **duplicate matches** ². In a duplicate match pair, the same cards are dealt twice with players swapped roles – drastically reducing variance. We may implement a simpler version: e.g., run two 500-hand matches, swapping which bot starts as Player 1 vs Player 2, and combine results. A fixed **number of hands** (e.g. 1000) per match is used to balance runtime with result confidence ⁶. We know from both competition and poker statistics that more hands yield more reliable win-rate estimates – thousands are needed; one might start with 1k-hand matches as in the MIT competition ⁶, but be prepared to run tens of thousands for tighter confidence intervals if needed ¹. If our variant is a tournament-style game (with fixed stacks and blinds), we should decide on a consistent structure (e.g., no escalating blinds mid-test, to keep each hand i.i.d. as much as possible). The harness might also allow different formats: perhaps a quick 100-hand test for smoke testing, and a full 5000-hand duplicate match for final evaluation.
- **Metrics Collection:** The harness should compute key **evaluation metrics** (Question 3) after each match or batch of matches. Primary metrics: **Win rate** (percentage of matches won, if playing a match-based format) and **average chip EV** (e.g. big-blinds per hand won on average). The engine's gamelog can be parsed to get final stack outcomes for each hand and thus cumulative winnings. We should also track **standard deviation/variance** of outcomes across matches – this helps determine if an observed improvement is statistically significant or just luck. With variance known, we can compute confidence intervals for the win rate difference. If possible, we may also evaluate **exploitability**: this is more complex (it requires computing how much worse the bot does against a theoretical best responder), but we can approximate it by testing the bot against a range of opponent styles (e.g., a very aggressive baseline, a very tight baseline) to see if it can be exploited by any particular strategy. A decrease in performance against a specific fixed opponent might indicate a strategy hole. All these metrics will feed into our plan.md priority decisions – e.g., if a new version shows higher EV but also higher variance, we might interpret that differently than a stable modest EV gain.
- **Resource Constraints & “No Pondering”:** The harness must respect the competition-like resource limits even though it's offline (Question 6 and Q12). That means each bot should only consume its allowed CPU time per hand/match – the engine's timing enforcement (or a wrapper timer in approach 2/3) will simulate this. Memory limits (e.g. 1 GB) should also be observed; for instance, if using containers, we can cap memory, or at least monitor it. The bots are not allowed to **ponder** (compute) outside of their active turns – in approaches 1 and 3, this is naturally enforced by the game engine (the bot processes only receive info on their turn and must respond within time, otherwise the engine can enforce a fold). In approach 2, we must be careful not to inadvertently give a bot extra computation between actions. As for incorporating harness results into automated tuning (Q12), we must do this **between matches**, not during. For example, one might use the harness to generate performance data for a set of parameter tweaks and then offline select the best parameter set (update the bot code accordingly). But we would not allow the bot to self-adjust during a match based on outcomes (that would violate no pondering and also break determinism). Essentially, any learning loop (like reinforcement learning through self-play) should treat each match as an episode and only update the strategy between episodes, producing a new static bot version

that can then be tested fresh. This way, each match still involves fixed agents with no online adaptation.

Having outlined the approaches, we next summarize them in a comparison table and then recommend the best path forward.

Comparison of Harness Approaches

The following table compares the three approaches across key dimensions:

Criteria	Process-Orchestrator (Engine)	In-Process Simulation	Client-Server / Container
Reliability (Stability of runs, isolation of failures)	High: Each bot in separate process; a crash or hang can be detected and handled without crashing the harness ⁴ . Engine manages game flow, reducing risk of logic errors.	Medium-Low: All bots share one process; an error in one can halt the whole simulation. Must trust bot code to behave. Harder to enforce time/memory limits internally.	High: Bots are fully isolated (possibly on separate hosts). Server can detect disconnects or rule violations and continue. However, network issues can interrupt matches if not handled.
Ease of Implementation	Moderate: Can reuse existing engine (with config for bots) – a lot of logic is pre-built ⁶ . Need to script launching matches and parsing results. Must handle I/O channels and process management.	Easy for simple tests, but Harder to fully implement: Quick to call functions if game logic is available. However, implementing complete game rules for a custom variant or hooking into engine code could be complex. Fewer moving parts, but more custom code for rules and timers.	Complex: Requires setting up a communication protocol and possibly container orchestration. More initial overhead (server code, networking). Running multiple matches might involve managing multiple processes/containers anyway (similar complexity to Approach 1 plus network layer).

Criteria	Process-Orchestrator (Engine)	In-Process Simulation	Client-Server / Container
Fairness & Determinism (Controlling randomness, equal conditions)	Good: Can set a fixed RNG seed in the engine for reproducibility ³ . Engine can alternate dealer positions each hand or we can run duplicate matches for fairness. Follows official rules exactly, so no inadvertent bias.	Good in theory: We can control the RNG easily and design the simulation to alternate seats. But if our implementation deviates from official rules, it could bias results. Need to be careful replicating all game mechanics (e.g. card dealing, “toss” mechanics) exactly.	Good: A custom server can be coded to deal deterministically with a given seed and enforce seat rotation or duplicate matches. However, synchronizing the same seed across a distributed system requires care. If using existing server software, we must trust its fairness implementation.
Security & Isolation (Preventing info leakage, cheating)	High: Memory and CPU space separated per bot. No shared globals. Bots communicate only via engine I/O. Hard for a bot to snoop on the other’s private data. The engine can also restrict bots’ file access (if run in sandboxed environment/user) and there is zero internet access.	Low: All code runs together – a malicious bot could theoretically introspect objects or override functions. Even without malice, shared process means accidental interference (e.g. if bots use the same pseudo-random generator without separate seeding, one bot’s calls could affect the other). Isolation measures (like running each bot in a thread with restricted permissions) in Python are weak.	Very High: Each bot in a container or separate OS process possibly on different machines – no shared memory or filesystem. Communication is only via sockets under strict protocol. One bot cannot read the other’s data or affect its execution except by in-game actions. This is the closest to a “hermetic” test environment.

Criteria	Process-Orchestrator (Engine)	In-Process Simulation	Client-Server / Container
Performance (Overhead, speed of running many games)	Moderate: Some overhead for inter-process communication and context switching, but generally the engine and bots run at native speed. For thousands of hands this is usually fine. The engine can be optimized (e.g., compiled eval7 library for card eval). Running many matches sequentially is straightforward; running many in parallel is possible but limited by CPU cores if all on one machine.	High Speed: No IPC overhead, just function calls. Simulations can run as fast as the code allows (could potentially simulate thousands of hands per second if bots are simple and logic is efficient). Great for large-scale self-play experiments in a single process. The limitation might be Python's speed for heavy logic, but still likely faster than coordinating processes. Parallelism is limited by GIL (for Python), but one process could simulate one match at a time quickly.	Lower: Network communication adds latency (though on localhost it's small). If containers are on one machine, context switching overhead is similar to Approach 1 plus a bit more. If distributed, network latency could slow things. Running in parallel is feasible by distributing bots to different machines or launching multiple server instances, but complexity increases. Overall throughput may be lower unless scaled out.

Criteria	Process-Orchestrator (Engine)	In-Process Simulation	Client-Server / Container
Logging & Observability (Insight into game and bot decisions)	Rich: Engine can produce a detailed game log file ⁷ . We can also capture each bot's output logs (if they have any) up to a size limit ⁷ . The harness controller can compile stats from these logs. Observing live progress is possible (engine could print updates each hand if verbose). However, introspecting a bot's internal state in real-time is not possible (since it's out-of-process, we rely on what it logs or post-match analysis).	Rich (with access): Since everything is in one process, the harness can directly query bot objects for internal info at any time, or step through decision functions. This is great for debugging logic errors or extracting additional info (e.g., we could instrument the bot to output its equity estimate at each decision). We have to implement logging ourselves, but it can be as detailed as needed (structured data in memory, etc.). The downside: too much logging or debugging can slow down the simulation. Also, we must ensure that any introspection doesn't inadvertently give one bot knowledge of the other (maintain the integrity of the game).	Decent: The server can log all actions and results. If we control the server code, we can add hooks to log things like time taken per decision, pot sizes, etc. Each bot can keep its own logs within its container. Real-time observation is harder – we might rely on the server's view of the game. If we wanted to see inside a bot's decision process, we'd have to either instrument the bot's code to output debug info or attach a remote debugger to that container. So, while logging game events is straightforward, deep introspection is limited compared to in-process.

Compatibility & Versioning

(Using historical versions, integrating into dev workflow)

Good: We can point the engine to different bot executables or scripts for Player 1 and Player 2. Storing old versions is as simple as keeping old bot directories or binaries (or using source control tags and rebuilding if needed). Because the interface is standardized (e.g., each bot reads game state from stdin and prints actions), even an older bot will work as long as that protocol hasn't changed. If the protocol did change with a new variant, we might need to include a compatibility layer or run it under the older engine in tandem. For safety, we might **snapshot** each bot version – e.g., store a copy of its code and dependencies in a timestamped folder – so we can always rerun it later under identical conditions. Integration into dev workflow: we can automate matches as part of a CI pipeline (run a quick self-play after each commit) or schedule nightly regression tournaments between latest and baseline. The process approach is script-friendly (just call the engine CLI with appropriate config).

Mixed: If the bot code is easy to import, we can load older versions (maybe by importing from different paths or using Git to check out an older commit in-memory). But code changes might make older versions incompatible in the same run (for instance, if the bot's API changed). One workaround is to containerize or virtualenv different versions and then use process approach – which is basically falling back to Approach 1. So maintaining many historical versions in one codebase is tricky here. The in-process method shines more for testing new strategies quickly rather than long-term version regression, unless we maintain a backward-compatible interface for all versions. In terms of dev workflow, this approach could be part of unit tests (simulate specific hand scenarios to ensure new logic behaves as expected compared to old logic). But it's less suited to full automated tournaments between dozens of versions – that would get unwieldy in one process.

Good, but heavy: By design, having each bot as a separate deployed entity (container or client program) means we can archive versions easily (each version is an image or binary). Compatibility issues are minimal if each container includes the correct environment for that version. We could spin up any two versions and have them connect to the server. This is basically how official competitions handle entries. For integration, we could use a container orchestration (like docker-compose or similar) to bring up a match between any specified versions. It's automatable, but requires maintaining possibly many container images. Versioning is very explicit (which is good for clarity, but more storage). If our goal is to frequently test incremental changes, building a new container each time is slower than Approach 1's method of just calling a new script. So this approach might be better when testing a few significant versions rather than every small commit.

Table Key Takeaways: Approach 1 (process engine) and Approach 3 (client-server) both provide strong isolation and reliability, with Approach 1 being simpler to implement for our offline needs. Approach 2 is lightweight and flexible for debugging but poses risks in reliability and fairness. Given these comparisons, the **process-based engine harness emerges as the most practical solution** for a robust regression framework, combining good security, determinism, and reasonable ease of use.

Recommended Approach and Justification

After evaluating the options, **we recommend implementing the self-play regression harness using the External Process Orchestrator approach, leveraging the official Pokerbots engine as the match runner.** This approach best meets the project requirements and addresses the key questions, for the following reasons:

- **Proven Reliability:** The official engine already implements the core match mechanics and enforces rules identical to the competition. By using it to run bot-vs-bot matches, we inherit its robustness and safety features (e.g. subprocess management, timeouts) ⁵. This dramatically reduces the chance of our harness failing due to a bug in game logic or a bot freezing the system. We can trust the engine to correctly deal cards, apply the “Toss or Hold’em” variant rules, and determine winners, which eliminates a whole class of potential errors on our side.
- **Isolation & Fair Play:** Running each pokerbot in a separate process ensures a **fair fight** with no unintended information sharing ⁴. Especially given the constraint of **no internet and no pondering**, this isolation guarantees that bots only react to the official game info (no sneaky shared memory or background threads gathering extra data). It also means we can pit *any* combination of versions or even different bots easily – e.g., current version vs last week’s version, or our bot vs a known baseline AI – by just pointing the engine to different program paths. The engine’s standardized interface means each bot gets the same environment (same initial stack, blinds, etc. from config) ⁶, upholding fairness.
- **Determinism and Regression Testing:** With a small modification, we can make matches reproducible by controlling the engine’s RNG seed. This is straightforward to implement (for example, seed the `eval7` library used by the engine with a fixed value ³). Doing so allows us to rerun the exact same sequence of cards for two different bot versions, isolating the cause of any difference in outcome to the bot decisions. This is invaluable for regression testing (Question 4) – if a new code change causes a loss where the old version would win, we can replay that specific scenario to diagnose the mistake. No other approach offers this combination of realism and reproducibility as cleanly.
- **Comprehensive Metrics & Logging:** The engine can produce detailed logs of each game, which we will utilize to compute our performance metrics. By parsing the gamelog ⁷, we can calculate win rates, EV per hand, standard deviation, etc. The engine’s built-in logging and our harness scripts on top will also allow us to capture any custom info we need (for instance, we could instrument our bot to output its internal evaluations to stderr, which the harness can record for post-game analysis, staying within the log size limit ⁷). This addresses Question 3 and 7 by giving us the data needed to evaluate improvements and attribute outcomes to specific actions. If we want to analyze specific decisions (say, why the bot chose to toss a certain card), we’ll have the full action trace and even

could add an “equity dump” in the bot that the harness records when enabled. Using the engine ensures that the log of actions is accurate and formatted consistently, simplifying analysis.

- **Statistical Rigor (Match Formats):** The recommended harness can easily implement robust match formats to reduce variance (Question 5). For example, we can run automated **duplicate matches**: first run the engine for N hands with Bot A as Player 1 and Bot B as Player 2, then swap their positions and replay the **same N hands** (by reusing the random seed and just swapping the bot command paths). Combining the results cancels out card distribution luck – any consistent outperformance is real ². This approach is directly enabled by the engine harness; doing the same in an in-process setup would require a lot of manual state management. Additionally, the engine’s configuration allows a large number of rounds per match (e.g. 1000) ⁶, and we can script running multiple matches in a series (e.g. 20 duplicate pairs) to gather as many hands as needed. The harness can thus be configured to trade off statistical confidence vs. runtime: quick smoke tests (1 match of 1000 hands) and thorough evaluations (tens of thousands of hands across many matches). The process approach makes it straightforward to parallelize matches too (spawn multiple engine instances on different CPU cores for independent trials) if we need faster results – something that pairs well with our offline, no-internet scenario where we might have a single powerful machine at our disposal.
- **Security & Sandbox:** While full container isolation was considered, it’s not strictly necessary for our offline dev/test context and would add overhead. The process approach already prevents a bot from tampering with another. We can enhance security by running the bot processes with restricted permissions if needed (e.g., a chroot or limited Linux user that can’t open disallowed files), but given “no internet” and well-behaved bots, this is mostly a precaution. The crucial point is that process isolation prevents any accidental cross-talk or dependency conflicts between versions. Each bot runs from its own directory (we will store historical versions separately), so there’s no chance of, say, an old bot accidentally importing the new bot’s code or vice versa. This containment is a big advantage for apples-to-apples comparisons.
- **Handling Failures Gracefully:** In the recommended setup, common failure modes are easier to manage. If a bot exceeds the time limit, the engine can automatically fold it or end the match (we’ll see it in the log and treat it as a loss for that bot). If a bot process crashes or exits, the engine detects that the pipe closed and can award the win to the opponent. No deadlock will freeze our entire harness; at worst one match ends early and we record the failure. Our harness wrapper can be designed to catch any non-zero exit codes or timeouts and log them as “bot X failed in match Y due to timeout or error”. We can then prioritize fixing those failures. In contrast, in an in-process design, a crash would likely stop the whole run. Thus, the process engine approach is inherently more **fault-tolerant**. We will of course test the harness itself by simulating a timeout or injecting a bug to ensure it handles such cases (maybe by creating a dummy bot that deliberately stalls to see that the engine cuts it off).
- **Integration into Plan & Development Cycle:** By making this harness Priority 0, we ensure that every subsequent bot change is evaluated objectively. Concretely, we will integrate harness usage into our **plan.md** workflow as follows:
 - When a new bot version is developed (or a new feature branch), run a batch of self-play matches against the current champion bot.

- Collect the metrics (win rate, EV, etc.) and update a **scoreboard** or report in plan.md. For example, “Version 1.2 vs Version 1.1: +0.05 BB/hand (+55±30 over 20k hands) – statistically insignificant, need more data” or “Feature X branch vs current: -2% win rate over 5k hands – indicates regression, do not merge.”
- This harness-driven feedback will directly inform priority: improvements that show clear positive results will be promoted, while those that don’t will be reworked or deprioritized. Over time, we’ll build a **history of version performance**, which helps us track progress and avoid reintroducing past bugs.
- Since the harness uses offline computation only, it doesn’t violate “no pondering” – all this evaluation happens during development, not when the bot is live in a tournament. The final submitted bot remains static. If we pursue automated tuning (like hyperparameter optimization or self-play reinforcement learning), the harness results can close the loop: e.g., run a set of matches to evaluate a new strategy, update the strategy parameters offline, and then validate again. This is essentially how AlphaZero-style training evaluates new models, accepting a new model only if it wins >55% of games vs the old ¹⁰. We can employ a similar gating criterion in our plan: only promote a new version if it statistically significantly outperforms the current one (perhaps with a threshold like 55-60% win rate in tests) ¹⁰.

In summary, the process-based engine harness hits the sweet spot: **it’s robust, aligned with competition constraints, and relatively quick to set up** given we have the engine and bots ready. It will provide trustworthy data to guide our development. Approaches like an in-process simulation are useful for unit tests or when we need to peek inside the bot’s mind, and we can still use that in specific cases (for example, to debug why a certain decision was made, by replicating a situation in a Python test). But for the core regression and ranking framework, we stick with the battle-tested method. The client-server/container approach, while very secure, is unnecessary overhead for our use case – we’re not running a public server, just offline matches, and we prefer to invest time in improving the bot rather than maintaining complex infrastructure.

Thus, we will proceed to implement the harness by extending the MIT Pokerbots engine (or our local variant of it) to run automated self-play sessions between arbitrary versions, gather metrics, and output results for inclusion in plan.md.

Implementation Considerations & Checklist

To ensure a successful harness deployment, we outline the following checklist of implementation steps and best practices:

- **Preserve Historical Versions:** Establish a method to **version-control bot snapshots** for fair comparison. For each significant bot iteration, save a snapshot of the code (and any data files) in a separate directory or branch. For example, maintain a `bots/` folder with subfolders `v1.0`, `v1.1`, etc., each containing a runnable bot. This guarantees we can always reproduce past bots’ behavior even if the main codebase moves on. Document the exact commit or hash corresponding to each version in plan.md. If using Git, consider tagging releases (e.g., `git tag v1.0`) and be able to build that version on demand. **No changes** should be made to old versions except what’s needed to interface with the harness (and that should be standardized for all). This addresses versioning (Q2) and compatibility – essentially freezing the bots in time.

- **Engine Configuration:** Set up the poker engine for harness use. In the `config.py`, enable deterministic settings where possible (if the engine allows seeding via config or by modifying its init to accept a seed). Configure match length and structure: e.g., `NUM_ROUNDS` to a default (like 1000 or more) ⁶, and ensure **ENFORCE_GAME_CLOCK = True** with appropriate time bank (e.g., 30s) to simulate timeouts ⁵. Disable any randomness in opponent seeding or seat assignment by manually controlling it (perhaps always start Bot A as Player 1 for the first match, then we will swap in code for the second match to ensure fairness). If the variant has any stochastic elements beyond dealing (like random card toss-ups), make sure those draw from the same RNG so the seed covers them too.
- **Harness Orchestrator Script:** Develop a top-level Python script (or use a Makefile/Batch if preferred) that **runs matches in bulk**. This script will take as input two bot versions (or a list of matchups) and automate: (a) writing the engine config to point to the two bot directories (the fields `PLAYER_1_PATH`, `PLAYER_2_PATH` in config) ¹¹, (b) setting a random seed if possible, (c) launching the engine (e.g., `subprocess.run(["python3", "engine.py"])`) and waiting for it to finish, (d) parsing the results (gather final chip counts or win/loss from the gamelog), and (e) repeating for the reversed matchup (to implement a duplicate pair). It should accumulate statistics over multiple runs if needed. Include robust error handling: if the engine process returns an error or if a bot crashes (which might be indicated in the gamelog or by non-zero exit code), record that event. Possibly implement retries if a match doesn't complete, but also flag it for investigation.
- **Logging and Data Collection:** Design a **logging schema** that is both minimal and sufficient (Q7). The engine's `gamelog` will log every hand's actions – we will keep those files for deep debugging but they might be too verbose to include entirely in plan.md. Instead, have the harness produce a **summary report** per match or per series: e.g., "BotA vs BotB: BotA won X out of Y matches (Z hands played) with average +M chips." and maybe "significant at p-value P". For each match, log the key outcomes: number of hands played (should equal config `NUM_ROUNDS` unless someone busted earlier), final stack of each player, and any rule infractions (timeouts, illegal move if any). If a bot exceeded time, note "BotB timed out at hand 123". Also log the seed used for that match for traceability ("Match seed: 42"). For minimal attribution logging, capture critical decisions: you could log something like a **hand trace ID** if something went wrong. For example, if we detect a large sudden loss, we might mark "Hand 57: BotA went all-in with 7-2 and lost – see gamelog for details". But in general, the summary focuses on metrics unless deeper analysis is needed. Ensure logs are written to distinct files or namespaces per match to avoid conflicts (the engine might write to `gamelog.txt`; if running multiple matches in parallel, give each a unique output file or run sequentially).
- **Metric Computation:** After collecting raw results, compute the **evaluation metrics**. Automate calculation of: win-rate (if we define a "win" as finishing with higher stack or busting the opponent in a tournament format), average **EV (expected value) per hand** in terms of big blinds or chips (this could be total chips won divided by hands played, which the competition often uses, e.g., small bets/hand ¹² ¹³). Also compute the **standard deviation** or standard error of the EV across matches or across duplicate pairs. If running many duplicate matches, we can derive an empirical distribution of outcomes. Use this to get confidence intervals. For example, if BotA beat BotB in 18 of 20 duplicate matches, that's a strong sign; if it's 11 of 20, that's inconclusive statistically. We might incorporate a statistical test (like a t-test or a non-parametric test for win-rates) to decide if one version is significantly better. These calculations can be output in the summary. Possibly maintain an ongoing

Elo rating for versions as well (there are known methods to convert heads-up poker results to Elo-like ratings). This can help track progress over time.

- **Variance Reduction Techniques:** Implement the **duplicate match pairing** in the harness. For every matchup test, ensure we run the match with players in one seating, then swap and run again with the *same deals*. We will likely need to hack the engine to reuse the shuffle. One way: run the first match normally (engine will shuffle internally). Extract the sequence of deck(s) used from the gamelog (if the engine doesn't output the exact deck order, an alternative is to log the engine's RNG seed, or instrument the engine to output the seed at start). Then modify the engine to accept a preset deck or seed for the second run, and just swap the bots' assignment. Alternatively, we start both engine runs with the same seed value (if seeding at start leads to identical deals) and swap bot order for the second – since RNG sequence will be identical, the deals should mirror, giving the duplicate effect ². Verify this works correctly. If duplicate matches are too hard to coordinate, at least ensure an even number of hands where positions alternate each hand (the engine likely already alternates blinds each hand in heads-up). Either way, emphasize **equal opportunity** – each bot should play from each seat equally in the test.
- **Time Management & No-Pondering Enforcement:** Test that the time controls are effective. For example, deliberately insert a `sleep` in a bot to exceed the time and see that the engine logs a timeout and moves on. The harness should catch that event and treat it as a loss for that bot. Also ensure that bots cannot act outside of their turn – the engine should ignore any extraneous output from a bot. “No pondering” is inherently enforced by the engine's model (a bot only gets input on its turn). To further enforce memory limits or CPU, if possible run the bot processes with constraints (on Linux, we could use `ulimit` for memory or nice levels for CPU). This might not be necessary if the competition doesn't strictly enforce beyond time (often if a bot uses too much memory and crashes, that's on the bot). But as a harness, we can observe memory usage to catch any looming issues (maybe log peak RSS if easy to get). Essentially, incorporate resource usage monitoring in the harness for insight, without affecting the match unless a limit is exceeded.
- **Sandboxing & Cleanup:** For each match run, isolate bot working directories if needed. For example, if a bot writes a log or uses a temp file, direct those to separate locations (so two bots don't write to the same file name). Clean up any leftover processes or files after matches. If using network sockets (in case we adapt a server mode), ensure ports are freed (maybe add a short delay or reuse socket options). Since we plan to use subprocess directly, just make sure to terminate processes if the engine doesn't terminate on its own (maybe the engine closes when match over, which it should). The harness script should not hang waiting for input that never comes – implement timeouts at the script level too (e.g., if a match hasn't finished in, say, 60 seconds which is double the allotted time, kill it and mark as failure).
- **Integration with plan.md:** Define how the harness outputs will be presented in our planning document. Perhaps maintain a section in plan.md for “Regression Test Results” that automatically (or manually with copy-paste) gets updated after each major change. This section might list recent head-to-head results: “Bot v1.3 vs v1.2: +0.10 EV/hand (statistically significant at 95% CI) – improvement observed, promote v1.3 as new baseline ¹⁰. Key changes: improved bluff strategy.” or “v1.4 vs v1.3: -0.05 EV/hand (not significant) – no clear improvement, further tuning needed.” We should also highlight any *exploitability tests* here: e.g., “v1.3 vs AggressiveBot: win rate 52% (down from 60% in v1.2) – suggests new strategy is more exploitable by aggression; consider

countermeasures.” This will tie the harness data directly into priority decisions (if a change makes us worse against a certain opponent, maybe priority to fix that). Automating the editing of plan.md could be complex, so initially we can manually update it with results from harness runs. The key is to ensure this feedback loop is consistently maintained.

- **Failure Mode Documentation:** Keep a log of any harness failures and how we mitigated them. For instance, if we encounter a deadlock (say the engine waiting on a bot that never connected), note that and adjust (maybe increase `CONNECT_TIMEOUT` if needed ¹⁴). If we see I/O stalls (one bot printing huge data slowing things), consider truncating or ignoring bot outputs beyond a limit – the engine config already sets `PLAYER_LOG_SIZE_LIMIT` ¹⁵ which helps. Our harness should be resilient: if one match fails, it should move on to the next rather than aborting the whole batch. This might involve try/except around the match execution. We'll output a warning in the summary for any match that failed and continue. Over time, as the bots stabilize, these issues should be rare, but the harness should be prepared for them (especially when integrating new, possibly bug-ridden versions for testing).
- **Optional: Automated Tuning Loop:** If we venture into automated self-play improvements (e.g., hyperparameter tuning or CFR-style strategy iterations), we can integrate that with the harness carefully. For example, we could generate a new candidate strategy, use the harness to play it against the current best for X matches, and if it wins >55% ¹⁰, accept it as the new best – exactly how AlphaZero/ELF OpenGo did. This would all happen offline. We must ensure that the self-play games used for training are separate from the evaluation games (to avoid bias). We likely won't implement full reinforcement learning within the competition timeline, but even heuristic tuning (like trying different aggression factors) can be guided by harness results. Just keep any learning strictly out-of-match. The harness might dump data that a training script uses (like frequencies of certain actions succeeding, etc.), but the bots running in matches should not adapt mid-match.

By following this checklist, we will create a robust self-play regression harness that not only answers all the key questions but becomes an integral tool for continuous improvement of our pokerbot. The end result will be a well-tested, empirically validated pokerbot, with a clear record of how each change affected performance – a crucial asset in a competitive development environment.

Bibliography & References

- **MIT Pokerbots Engine Docs (2024):** Official engine reference for Texas Hold'em. Demonstrates launching bots as subprocesses and configuring match parameters ⁵ ⁶. Key insight: built-in support for game clock and multi-round matches.
- **Poker Competition Formats:** AAAI 2006 Computer Poker Competition report explaining *duplicate match* methodology ² – dealing same cards twice with seat reversal to reduce variance. Informs our fairness approach.
- **Poker Variance & Sample Size:** Discussion on poker win-rate variance indicates needing ~50k–100k hands for statistical confidence in winrate ¹. Underscores our use of many hands and variance metrics.
- **AlphaZero/ELF OpenGo Self-Play Evaluation:** Facebook's ELF OpenGo paper notes using 400 games and requiring >55% win-rate to accept a new model ¹⁰. We adopt a similar threshold for promoting new bot versions.

- **Engine Determinism via Seeding:** Fedden's Poker AI project highlights seeding the random generator to make poker simulations deterministic ³. This supports our plan to fix RNG seeds for reproducible tests.
- **PyPokerEngine Framework:** Documentation of PyPokerEngine (Ishikota) showing how to run AI vs AI in-process and log game events ⁸ ¹⁶. Illustrates the simplicity of in-process simulation, though we chose the more robust external engine route.
- **MIT Pokerbots 2024 Config:** Configuration file snippet showing log file naming and size limit, as well as timeouts ⁷ ⁵. Reinforces the engine's built-in logging and timing controls which we leverage.
- **Engine Readme on Subprocess Safety:** MIT Pokerbots engine readme emphasizes incorporating improvements for safe bot subprocess handling ⁴ – validating our process-based approach for safety and isolation.

¹ How many hands until my winnings can be considered statistically significant : r/poker

https://www.reddit.com/r/poker/comments/cmmxhf/how_many_hands_until_my_winnings_can_be/

² ⁹ ¹² ¹³ martin.zinkevich.org

<https://martin.zinkevich.org/publications/competition.pdf>

³ GitHub - fedden/poker_ai: An Open Source Texas Hold'em AI

https://github.com/fedden/poker_ai

⁴ GitHub - mitpokerbots/engine: MIT Pokerbots Engine

<https://github.com/mitpokerbots/engine>

⁵ ⁶ ⁷ ¹¹ ¹⁴ ¹⁵ engine-2024/config.py at master · mitpokerbots/engine-2024 · GitHub

<https://github.com/mitpokerbots/engine-2024/blob/master/config.py>

⁸ ¹⁶ Introduction - PyPokerEngine

<https://ishikota.github.io/PyPokerEngine/>

¹⁰ [1902.04522] ELF OpenGo: An Analysis and Open Reimplementation of AlphaZero

<https://ar5iv.labs.arxiv.org/html/1902.04522>